THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Data Science Program

Project Report - Spring 2025
Group - 4

High-Performance Semantic Segmentation for Autonomous Urban Navigation

Pranjal Wakpaijan

Data Science Program

Columbian College of Arts & Sciences

Supervised By

Amir Jafari

Introduction

Image segmentation, especially semantic segmentation, is a powerful computer vision technique where each pixel in an image is assigned a class label. In autonomous driving, this task helps segment objects like roads, cars, pedestrians, and buildings. Traditional segmentation models often rely on convolutional neural networks (CNNs), but recent advances with Vision Transformers (ViTs) have shown promising results. ViT offers a unique approach by handling images as sequences of patches, similar to how Transformers in NLP treat words, which makes it especially powerful for dense prediction tasks like segmentation.

Why Use Vision Transformers for Image Segmentation?

ViTs are particularly effective for segmentation due to their ability to:
Capture Long-Range Dependencies: In urban scenes, it's essential to understand relationships between distant objects (e.g., a pedestrian and a car), which CNNs can struggle with due to local receptive fields.
Reduced Inductive Bias: ViTs are more flexible, as they don't impose strong spatial locality biases like CNNs. This flexibility allows them to adapt to complex structures in images better.
Scalability: ViTs can handle larger images with more tokens, enabling them to manage high-resolution images as required in segmentation tasks for driving scenarios.

Methodology

1. Data Loading and Class Balancing

The A2D2 dataset was loaded using a custom A2D2_CSV_dataset class, which processes image-mask pairs from a CSV file. To address class imbalance (e.g., frequent classes like "road" vs. rare classes like "pedestrian"), the dataset was split into training (70%), validation (15%), and test (15%) sets with stratified sampling. Images were resized to 1208x1920, with bilinear interpolation for images and nearest-neighbor for masks to preserve class labels.

Execution: Data loaders were created with multi-worker support and prefetching for efficient batch processing. Class balancing was initially handled implicitly through loss functions, but no explicit oversampling was implemented in the first iteration.

2. Utility Functions for Metrics

To evaluate segmentation performance, the following metrics were defined:

1.Mean intersection over union

The mIoU is used to measure the model's accuracies. The mIoU is defined as follows:

$$mIoU = \frac{1}{N}\sum_{n=1}^{N} IoU = \frac{1}{N}\sum_{n=1}^{N} \frac{TP}{(TP + FP + FN)}$$

Where TP, FP, and FN are the numbers of true positive, false positive, and false negative pixels determined over the whole test set. N is the number of the defined classes. [3] The evaluation demonstrates that semantic segmentation models achieve higher frame rates and accuracy values using higher image resolution.

2.Pixel Accuracy

An alternative metric to evaluate a semantic segmentation is to simply report the percent of pixels in the image which were correctly classified.[5]

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

3.Loss

The Segmentation Transformer model supports multiple loss functions, selectable through the args.loss parameter, to optimize training for semantic segmentation. Cross Entropy Loss (ce) serves as the standard loss for multi-class segmentation and is the default if no other is specified. Focal Loss focuses on hard examples with a focusing parameter gamma=2.0 and class balancing factor alpha=0.25. Dice Loss optimizes overlap between predicted and ground truth masks, using a smoothing factor smooth=1.0. Combined Loss is a weighted combination of Focal and Dice losses, with dice_weight=0.5 and focal_weight=0.5, balancing the strengths of both.

3. Model Architecture

The Segmentation Transformer was designed with the following components:

a. Patch Embedding

The Patch Embedding module is a critical component of the Segmentation Transformer, designed to transform input RGB images into a sequence of embeddings suitable for transformer processing. Its primary purpose is to split high-resolution input images (e.g., 1208x1920 for the A2D2 dataset) into smaller, manageable patches, which are then embedded into a fixed-dimensional space. This is achieved using a convolutional layer with a kernel size equal to the patch size (16x16) and a stride of 8, effectively extracting overlapping patches from the image. The resulting feature maps are flattened and transposed to produce a sequence of embeddings, each with a dimension of 768, enabling the transformer to process the image as a series of tokens, akin to words in natural language processing. This approach preserves spatial information while reducing the computational complexity of handling high-resolution images, setting the stage for the subsequent transformer encoder and decoder layers to capture global and local context for semantic segmentation.

b. Transformer Encoder

The Transformer Encoder is designed to process patch embeddings to capture the global context of an input image for semantic segmentation. It is implemented as a stack of 12

encoder layers, each comprising multi-head self-attention with 12 attention heads and a feed-forward MLP with an MLP ratio of 4.0. The encoder employs a pre-norm architecture, applying layer normalization before the attention and MLP sub-layers, and incorporates residual connections to enhance training stability. A dropout rate of 0.1 is used to prevent overfitting, enabling the encoder to effectively model long-range dependencies across image patches.

c. Transformer Decoder

The Transformer Decoder is tasked with refining the encoder outputs to generate segmentation features for the Segmentation Transformer. It is implemented with 4 decoder layers, each incorporating self-attention to process its own inputs and cross-attention to leverage the encoder's outputs as memory, enabling the integration of global context. The decoder's configuration matches that of the encoder, maintaining consistency in attention heads, MLP ratio, and other parameters, to effectively produce refined features for pixel-level class predictions.

4. Model Creation

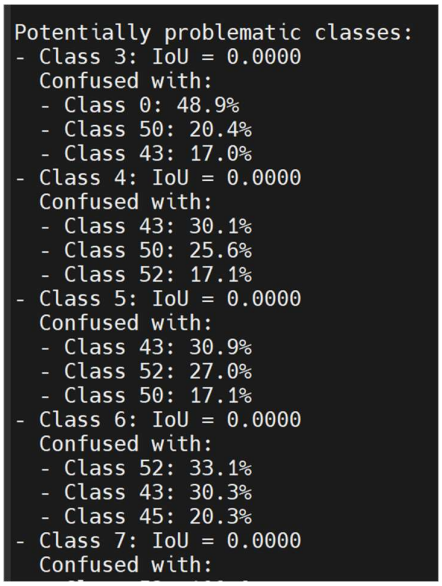
A create_segmentation_transformer function instantiated the model with configurable hyperparameters. A fallback to DeepLabV3-ResNet50 was included for robustness in case of memory issues.

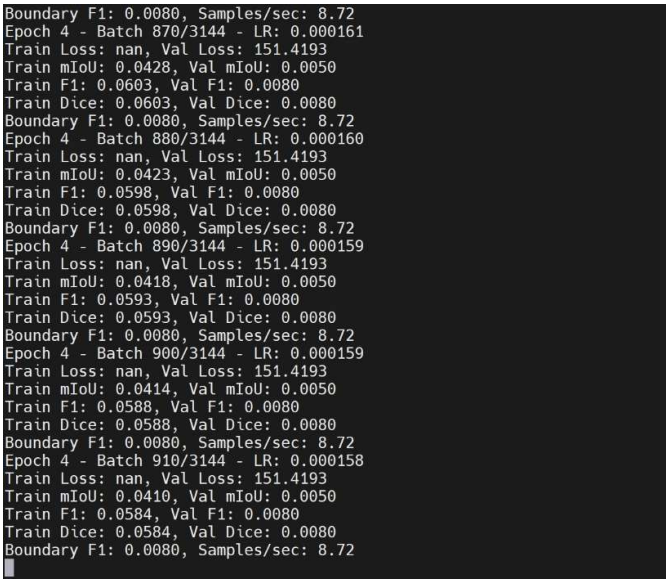
5. Custom Loss Functions

To address class imbalance and enhance segmentation performance, the Segmentation Transformer incorporated several custom loss functions, selectable via the --loss command-line argument, with Cross Entropy Loss as the default. Cross Entropy Loss served as the standard multi-class loss for segmentation tasks. Focal Loss was implemented to focus on hard examples, using a focusing parameter gamma = 2.0 and a class balancing factor alpha = 0.25. Dice Loss was designed to optimize overlap between predicted and ground truth masks, with a smoothing factor smooth = 1.0. Additionally, a Combined Loss was introduced, combining Focal and Dice losses with equal weights (dice_weight = 0.5, focal_weight = 0.5) to leverage the strengths of both approaches.

6. Training and Evaluation Logic

Setup: The model was moved to GPU, initialized with the AdamW optimizer (learning rate = 0.005, weight decay = 0.05), a custom learning rate scheduler with warmup, and mixed-precision training (AMP) for faster computation. Training Loop: It ran for 1 epoch by default, processing batches with loss computation, backpropagation, and weight updates, with validation performed every log_freq batches to monitor progress. Checkpointing: The best model was saved as best_model.pth based on validation mIoU, alongside epoch-specific checkpoints (model-epX.pth) and the latest model state (latest.pth). Test Evaluation: Final metrics, including mIoU and Dice, were computed on the test set. Execution: Training metrics were saved as JSON files in the checkpoint_dir for analysis.





7.Post-training analysis

Post-training analysis for the Segmentation Transformer encompassed several key components to evaluate and interpret model performance. Segmentation outputs were saved as predicted masks for the validation and test sets, with optional RGB color mapping for visualization. Attention maps were generated to visualize transformer attention patterns, enhancing interpretability, and saved as PNG files. Class-wise performance analysis was conducted to compute per-class metrics, identifying problematic classes with IoU below 0.5. A comprehensive Markdown performance report was produced, summarizing model details, evaluation metrics, and recommendations for improvement.

I faced some issues with the code when generating attention map, which later was addressed by my teammate.

8. Swin Transformer

To enhance performance, an attempt was made to replace the custom Segmentation Transformer with a Swin Transformer backbone [4], a hierarchical ViT designed for dense prediction tasks. The model initially ran, processing input images and producing segmentation outputs. However, it later failed and I have to switch bach to custom segmentation.

Results

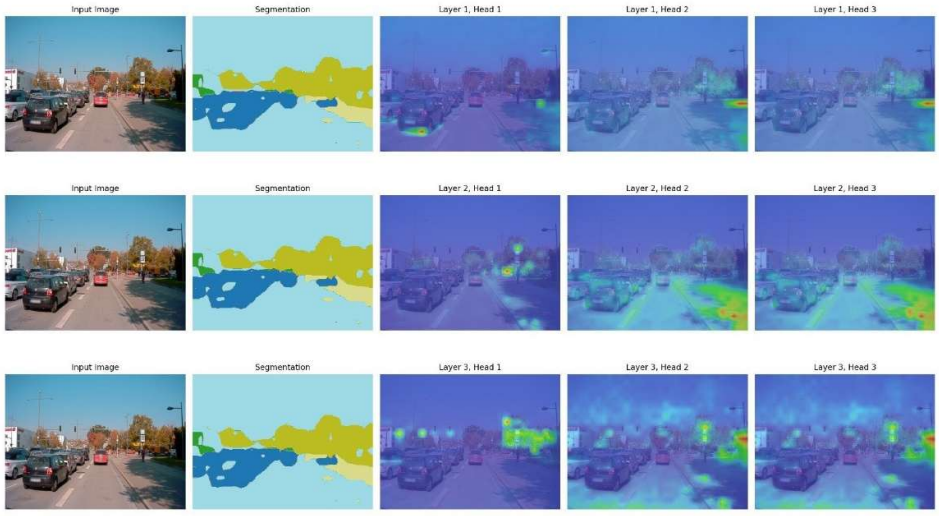
The Segmentation Transformer model underperformed significantly compared to the CNN-based architectures. It achieved a mean IoU of only 0.0908 and a pixel accuracy of 82.6%, indicating limited segmentation quality. Overlap-based metrics, including F1 score, Dice, and Boundary F1, were all around 0.12, while the recall (0.11) and precision (0.21) further highlight the model's struggle to detect and correctly classify objects. Despite the transformer's potential for global context modeling, the high loss value (0.4039) suggests

ineffective learning, possibly due to underfitting or insufficient training adaptation for dense prediction tasks.


```
{
  "loss": 0.4038801786822315,
  "mean_iou": 0.09083440899848938,
  "pixel_acc": 0.8260788321495056,
  "mean_recall": 0.11086513847112656,
  "mean_precision": 0.21368317306041718,
  "mean_f1": 0.12182522565126419,
  "mean_dice": 0.12182523310184479,
  "bf_score": 0.12182522565126419
}
```


The model and the output is saved in folder checkpoint.
Best Model: Saved as best_model.pth when validation mIoU improves.
Epoch Checkpoints: Saved as model-epX.pth.
Latest Model: Saved as latest.pth.
Metrics History: Saved as JSON files for training and validation metrics.

In the below figure generated by attention map,as we move from Layer 1 to Layer 3, the attention maps become more focused and localized, which is expected in a transformer architecture. Early layers (Layer 1) capture broad, global patterns, while deeper layers (Layer 3) refine these into more specific, object-centric features. However, the attention is still not precise enough, likely due to the model's limited training (one epoch) and underfitting.



Conclusion

The development and implementation of the Segmentation Transformer for semantic segmentation on the A2D2 dataset demonstrated the potential of Vision Transformers (ViTs) in capturing long-range dependencies and handling high-resolution images for autonomous driving applications. Despite the innovative design, including patch embedding, a robust transformer encoder-decoder architecture, and custom loss functions to address class imbalance, the model underperformed with a mean IoU of 0.0908 and a pixel accuracy of 82.6%, likely due to limited training with only a single epoch, which hindered convergence. The absence of explicit oversampling or weighted sampling may have further impacted performance on rare classes like pedestrians. Transformers, while powerful, require careful resource management for high-resolution datasets like A2D2, and leveraging pre-trained weights (e.g., from ADE20K) could significantly enhance convergence and overall performance. Future iterations should focus on extended training, explicit class balancing techniques, and pre-training to unlock the full potential of ViTs for dense prediction tasks in complex urban scenes.

References

[1] SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. [2105.15203] SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers

[2] Semantic Segmentation for Autonomous Driving: Model Evaluation, Dataset Generation, Perspective Comparison, and Real-Time Capability. [2207.12939] Semantic Segmentation for Autonomous Driving: Model Evaluation, Dataset Generation, Perspective Comparison, and Real-Time Capability

[3] Semantic Segmentation using Vision Transformers: A survey. Semantic Segmentation using Vision Transformers: A survey

[4] Swin-Transformer/models/swin_transformer.py at 2622619f70760b60a42b996f5fcbe7c9d2e7ca57 · microsoft/Swin-Transformer

[5] Evaluating image segmentation models. https://www.jeremyjordan.me/evaluating-image-segmentation-models/